

SENTINEL GUARD

Security & Requirements Report

CSG3101 Applied Project

May 2026

Table 0: Team Members

Team Member	Student No.	Responsibility	Report Section
Najath Mohamed Najeem	10686331	Web & Mobile Dashboard (D3)	Section E
Akila WAKISTA	10685843	Security & Encryption (D2)	Section B
Ravindu Sathsara	10694288	Backend & Server Comms	Section C
Julian Mendis	1068566	ML Classification (D6)	Section D
Methira Detumini	10698622	IoT Sensor Node (D1)	Section A

Contents

1. Introduction.....	4
1.1 System Architecture Overview.....	4
2. Requirements Traceability Matrix	5
Section A: IoT Hardware & Sensor Node	8
A.1 Hardware Components	8
A.2 Sensor Data Validation	8
A.3 Secure Communication (MQTTs)	9
A.4 Requirements Satisfied	9
Section B: Security & Encryption	10
B.1 Overview	10
B.2 Security Measurements.....	10
B.2.1 AES-128 CBC Encryption	11
B.2.2 HMAC-SHA-256 Integrity Verification	12
B.2.3 Replay Attack Detection	12
B.2.4 Key Management	13
B.3 Security Testing Results.....	14
B.4 Requirements Satisfied	14
Section C: Backend & Server Communication.....	15
C.1 Overview	15
C.2 Server Processing Pipeline	15
C.3 User Management API (FastAPI).....	16
C.4 Firestore Data Structure	16
C.5 Audit Logging.....	17
C.6 Requirements Satisfied	17
Section D: ML Classification & Anomaly Detection	18
D.1 Model Overview	18
D.2 Training Dataset.....	18
D.3 Feature Engineering	18

D.4 Model Evaluation Results	18
D.5 Mitigation Response	19
D.6 Requirements Satisfied	19
Section E: Web & Mobile Dashboard (Frontend)	20
E.1 Overview	20
E.2 Authentication & Role-Based Access Control.....	20
E.3 Dashboard Pages	21
E.4 Mobile Application	23
E.5 Requirements Satisfied	24
Section F: Non-Functional Requirements Evaluation	25
Conclusion.....	26

1. Introduction

Sentinel Guard is an IoT security system that protects environmental sensors and their communication from various attacks. These include middleman hacking, tampering with the payload, and replay attacks. The solution consists of hardware to collect the sensor data, multi-tiered cryptographic communications for protecting the data, machine learning algorithms that classify anomalies in the data, and an access control dashboard that can be accessed through web or mobile devices.

This report demonstrates that the Sentinel Guard product has been designed and built to fulfill both functional and non-functional specifications as outlined within the project proposal document. For all sections of this report, the system component is defined, and then we describe our design and implementation approaches and how the specific requirements have been met by each component of the Sentinel Guard system. If a requirement cannot be fully satisfied, we provide an explanation of the current status of the requirement and the reason it is not fully satisfied.

1.1 System Architecture Overview

The system operates across four layers:

- 1. IoT layer** - Utilizing an ESP32 microcontroller to interface with a DHT22 temperature and humidity sensor read measurements of the ambient conditions; then verify these measurements and securely transmit encrypted HMAC authenticated payloads of the sensor data via MQTTS (TLS 1.2, port 8883) to a Mosquitto broker located locally.
- 2. Security and backend layer** - A python application subscribes to the relevant MQTT topic, validates the received HMAC signature against the data to ensure the integrity of the message, checks the sequence ID of the message for replay protection, decrypts the content of AES-128 encrypted payload, and forwards only the valid sensor readings to the machine learning classifier. All verified sensor reading records are stored in FCM.
- 3. Machine learning classification layer** - Using the XGBoost machine learning model, each of the sensor readings is classified as either normal or anomalous. A record of each anomaly event that occurs is created in FCM's security_events collection. Only alert type mitigation methods are implemented; however, there are manual admin control mechanisms in place that can be using to invoke a much stronger intervention.
- 4. Front-end layer** - A React/Vite dashboard and a React Native Expo mobile app communicate in real time with FCM using Firestore's real-time data listeners to display live sensor data, log entries of security events, and classifications made by the ML classification system. Both applications authenticate with Firebase Authentication and employ RBAC methodologies to limit access to user accounts.

2. Requirements Traceability Matrix

Table 1 maps all functional and non-functional requirements from the project proposal to the system component responsible for satisfying them and their current implementation status.

Table 1: Requirements Traceability Matrix

ID	Requirement	Owner	Status
FR-6.1	Encrypt sensor payloads with AES-128 before transmission	B	Met
FR-7.1	HMAC-SHA-256 integrity verification of all incoming payloads	B	Met
FR-7.2	Reject payloads with invalid or missing HMAC	B	Met
FR-8.2	Replay attack detection via sequence identifiers	B	Met
FR-3.1	MQTT over TLS 1.2 (MQTTS) on port 8883	A / B	Met
FR-2.1	Validate sensor readings before transmission	A	Met
FR-1	Publish structured JSON payload to MQTT broker	A	Met
FR-15	Encrypted server-to-cloud communication (HTTPS/TLS)	C	Met
FR-09	Display ML classification result per security event	E	Met
FR-10.3	Admin mitigation controls (isolate node, block IP, reset keys)	E	Partial

ID	Requirement	Owner	Status
FR-11.1	Real-time alert notification to users on event detection	E	Met
FR-12.1	Security event detail displayed to administrators	E	Met
FR-13	Role-based access control across all interfaces	E	Met
FR-14	Authenticated session required for all dashboard access	E	Met
FR-15	TLS for all client - server communications	C / E	Met
FR-23.1	Audit log is accessible to administrators only	E	Met
FR-9	ML classifier identifies normal, anomaly, attack, false positive	D	Met
FR-10.1	Automated mitigation triggered on confirmed attack detection	D	Met
FR-19.1	User login via secure authentication service	E	Met
FR-20.2	User logout with immediate session termination	E	Met
FR-21.1	View historical sensor data	E	Met
FR-22.1	Acknowledge security alerts	E	Met

ID	Requirement	Owner	Status
FR-23.1	Full audit log export (admin only)	E	Met
NFR-01	System available 99%+ during demonstration period	A - E	Met
NFR-03	Sensor data collected every 3 seconds	A	Met
NFR-03	Sensor data delivered to dashboard within 3 seconds	E	Met
NFR-02	AES-128 and HMAC-SHA-256 cryptographic standards	B	Met
NFR-05	ML classification accuracy $\geq 90\%$ (overall F1)	D	Met
NFR-06	Automated mitigation within 3 seconds of detection	D	Met
NFR-7.1	Scheduled backup of data and audit logs	C	Pending
NFR-08	Dashboard compatible with major modern browsers	E	Met
NFR-9.1	Session timeout on inactivity	E	Met
NFR-10	Usable without specialist technical training	E	Met

Section A: IoT Hardware & Sensor Node

Responsible: Methira Detumina

Deliverable: D1

A.1 Hardware Components

An ESP32 microcontroller forms the basis of the sensor node to which a DHT22 temperature and humidity sensor is connected. The DHT22 is wired to the GPIO pin 4 and supplied power through the ESP32's 3.3 V output. The sensor node is assembled on a standard breadboard and supplied power through USB.

Table A-1: Hardware & Software Components

Component	Role	Detail
ESP32 Microcontroller	Central processing unit	Dual-core, Wi-Fi capable; manages sensor acquisition, validation, and MQTT publish
DHT22 Sensor	Temperature & humidity sensing	$\pm 0.5\text{ }^{\circ}\text{C}$, $\pm 2\text{--}5\text{ }\%$ RH accuracy; connected to GPIO pin 4 via 3.3 V supply
Arduino IDE	Firmware development	ESP32 board packages, WiFiClientSecure, PubSubClient, DHTesp libraries
Mosquitto MQTT Broker	Message broker	Configured for TLS on port 8883; topic: iot/sensor/dht22/secure

A.2 Sensor Data Validation

Each reading is checked against known physical limits before it is sent. The temperature reading must be between $0\text{ }^{\circ}\text{C}$ and $50\text{ }^{\circ}\text{C}$, and the humidity reading must be between 20 % and 70 % relative humidity. Each parameter's reading is validated independently; if one fails, the value is set to null and 'invalid' indicated in that parameter's status field. The valid parameter's data will still be sent. The overall packet_status will reflect the combined results as 'valid', 'partial_valid' or 'invalid'. This method allows for maintaining the integrity of all the sensor data, not allowing for a corrupted reading to propagate through the entire system.

A.3 Secure Communication (MQTTS)

MQTT over TLS 1.2 (MQTTS) is the protocol used for communication from the ESP32 to the Mosquitto broker on TCP port 8883. The CA certificate is built into the ESP32 firmware using the WiFiClientSecure library, which validates the broker's certificate during the TLS handshake. A sample capture from Wireshark show all traffic on TCP port 8883 is encrypted using TLS. The sensor readings are serialized into JSON, encrypted with AES-128, authenticated with HMAC-SHA-256 and published to the `iot/sensor/dht22/secure` topic.

A.4 Requirements Satisfied

- FR-3.1 – Connection to the Mosquitto broker via MQTTS on port 8883 with CA validation is established.
- FR-2.1– Sensor readings are checked against acceptable ranges for temperature (15 -35 °C) and humidity (20-70 %) before sending to the broker.
- FR-1 – The JSON message includes a `sequence_id`, sensor values, status and `packet_status`.
- NFR-3– Sensor readings are collected and sent to the broker periodically (e.g., every 3 seconds).

Section B: Security & Encryption

Responsible: Akila Wakista

Deliverable: D2

B.1 Overview

Sentinel Guard has a multi-layered security architecture that provides both transport and application level security solutions. At the transport level, the TLS 1.2 protocol provides secure channel access to devices. At the application level, all MQTT messages are protected through AES-128 encryption, HMAC-SHA-256 authentication, and sequence-numbered replay detection. Should the TLS channel be compromised but the MQTT message payloads are encrypted, HMAC-authenticated, and replay resistant.

B.2 Security Measurements

Table B-1: Applied Security Mechanisms

Mechanism	Purpose	Implementation Detail
AES-128 CBC	Payload confidentiality	Encrypts each MQTT message payload before it is published; unique IV's are added to each MQTT message payload to prevent identical ciphertext from occurring with identical plaintext.
HMAC-SHA-256	Integrity & authenticity	Each HMAC key is 160 bits; the HMAC digest is computed using the encrypted MQTT message payload. The ESP32 must compute an HMAC digest on the same payload and validate before it is processed.
Sequence ID	Replay attack prevention	Per MQTT packet, an incrementing integer is provided to identify the sequence number. The receiver tracks the last accepted sequence number from the sender and rejects duplicate or out-of-order sequence numbers when received.
TLS 1.2 (MQTTS)	Transport confidentiality	Port 8883; CA certificate is embedded in ESP32 firmware; all MQTT traffic will be opaque to network observers (Wireshark can confirm).

Mechanism	Purpose	Implementation Detail
Pre-shared keys	Key management	AES and HMAC keys are firmware-flashed during the time of deployment and will never be communicated over the network. A recommended best practice for AES and HMAC key management is to rotate every 90 days.

B.2.1 AES-128 CBC Encryption

Every sensor measurement is encrypted using AES-128 in CBC mode and serialised as JSON before being sent to an MQTT broker. Each encrypted message has a unique initialization vector (IV), meaning identical plaintexts will create different ciphertexts across multiple transmissions. This security design helps defend against attacks that rely on exploiting patterns of a ciphertext; the choice of AES-128 provides a solid option that meets the industry standard for encryption while being reasonable for the computational limitations of the ESP32 microcontroller.

Figure B.1

```

78 String encryptAESBase64(String plainText) {
79     const int blockSize = 16;
80
81     uint8_t padded[256];
82     uint8_t encrypted[256];
83     unsigned char ivCopy[16];
84
85     int plainLen = plainText.length();
86     int paddedLen = pkcs7Pad((const uint8_t*)plainText.c_str(), plainLen, padded, blockSize);
87
88     memcpy(ivCopy, AES_IV, 16);
89
90     mbedtls_aes_context aes;
91     mbedtls_aes_init(&aes);
92     mbedtls_aes_setkey_enc(&aes, AES_KEY, 128);
93     mbedtls_aes_crypt_cbc(&aes, MBEDTLS_AES_ENCRYPT, paddedLen, ivCopy, padded, encrypted);
94     mbedtls_aes_free(&aes);
95
96     unsigned char base64Output[512];
97     size_t base64Len = 0;
98
99     int rc = mbedtls_base64_encode(base64Output, sizeof(base64Output), &base64Len, encrypted, paddedLen);
100     if (rc != 0) {
101         Serial.print("Base64 encode failed, rc=");
102         Serial.println(rc);
103         return "";
104     }
105
106     String result = "";
107     for (size_t i = 0; i < base64Len; i++) {
108         result += (char)base64Output[i];
109     }

```

AES encryption workflow for MQTT payload protection

Note. This figure illustrates the process of converting validated sensor data into encrypted ciphertext using AES-128 before MQTT transmission.

B.2.2 HMAC-SHA-256 Integrity Verification

For securing the MQTT packets, once the sensor measurements have been encrypted, an HMAC digest is created by hashing the encrypted payload and included in the MQTT packet using a separate 160-bit HMAC key. The server then recomputes the HMAC after receipt and compares it against the HMAC digest received. Any alteration to the payload will yield a different HMAC, which will cause the packet to be rejected at the server before decryption. This process ensures data integrity and authenticity independently from the TLS channel.

Figure B.2

HMAC-SHA-256 verification process

```
try:
    expected_hmac = compute_hmac_hex(ciphertext)
    hmac_valid = (expected_hmac == received_hmac)
except Exception as e:
    upload_security_event("processing_error", f"HMAC computation error: {e}", seq, raw_text)
    return

if not hmac_valid:
    print("ALERT: HMAC VERIFICATION FAILED")

def compute_hmac_hex(data):
    h = HMAC.new(HMAC_KEY, digestmod=SHA256)
    h.update(data.encode("utf-8"))
    return h.hexdigest()
```

Note. This figure demonstrates how HMAC validation is used to verify packet integrity and authenticity before decryption.

B.2.3 Replay Attack Detection

Also, each packet will have a `sequence_id` field with an incrementing number. The server maintains the last successfully received sequence number and will reject any packets with duplicate or lower sequence numbers. This process helps to prevent replay attacks, as a legitimate packet will not be processed again, regardless of the validity of the HMAC and encrypted payload.

Figure 5.3

Replay attack prevention using sequence identifiers

```
ALERT: REPLAY ATTACK DETECTED
-----
Received Sequence ID : 1
Last Accepted ID      : 1678
Attempt Count         : 1/5 in 10s window
Mitigation            : Packet rejected
-----

SECURITY EVENT UPLOADED
Event Type   : replay_attack
Message      : Replay detected. Seq 1, last accepted 1678. Attempt 1/5.
Sequence ID  : 1
```

Note. This figure demonstrates how duplicate or outdated packets are detected and rejected by the receiver.

B.2.4 Key Management

Two symmetric pre-shared keys are utilized for the encryption/decryption of the payload using AES-128, and for generating and verifying the digest with an HMAC which is a total of 160 bits. The two keys remain separate, so that the confidentiality and integrity mechanisms are not cryptographically dependant upon one another. Prior to deployment, both keys are installed into both the ESP32 and the server-side receiver application via firmware flashes, and are not transmitted over a network. This demonstration utilizes static keys, while in a production deployment automated key rotation every 90 days is recommended using a secure out-of-band channel.

Figure B.4

Shared-key security model used in Sentinel Guard

```
// AES-128 key and IV
String encryptedBase64 = encryptAESBase64(plainJson);
String hmacHex = computeHMAC(encryptedBase64);
};

const unsigned char AES_IV[16] = {
    'H','G','F','E','D','C','B','A',
    '8','7','6','5','4','3','2','1'
};

// HMAC key
const unsigned char HMAC_KEY[] = "my_hmac_secret_2026";
```

Note. This figure illustrates the symmetric AES encryption key, initialization vector (IV), and HMAC authentication key configured in the ESP32 sensor node for secure MQTT communication.

Figure B.5
Server side shared-key verification and decryption process

```
# CRYPTO CONFIG
AES_KEY = b"12345678ABCDEFGH"
AES_IV  = b"HGFEDCBA87654321"
HMAC_KEY = b"my_hmac_secret_2026"

expected_hmac = compute_hmac_hex(ciphertext)
plain = decrypt_aes_base64(ciphertext)
```

Note: This figure shows the python MQTT server using the same shared AES and HMAC keys to verify packet integrity and decrypt encrypted sensor data received from the ESP32 sensor node

B.3 Security Testing Results

Table B-2: Security Validation Tests

#	Test	Outcome	Result
1	AES payload encryption	Wireshark captured traffic on port 8883 showed application data as opaque TLS records with no plain text MQTT content visible.	Pass
2	HMAC tamper detection	Manually modifying an encrypted packet in-transit allowed the server to correctly identify an HMAC mismatch and therefore reject the packets.	Pass
3	Replay attack detection	By re-injecting previously captured packets it was possible for the server to detect duplicate sequence IDs and thus block the processing of those packets.	Pass
4	Decryption verification	All valid packets were successfully decrypted at the server with their original JSON format intact.	Pass

B.4 Requirements Satisfied

- **FR-6.1:** AES-128 CBC encryption applied to every MQTT payload before transmission.
- **FR-7.1:** HMAC-SHA-256 digest computed and transmitted with each packet; verified server-side.

- **FR-7.2:** Packets with invalid HMAC are rejected before decryption or processing.
- **FR-8.2:** Duplicate and out-of-order sequence IDs detected and blocked.
- **FR-3.1:** Application-layer security applied in addition to TLS (shared with Section A).
- **NFR-02:** AES-128 and HMAC-SHA-256 cryptographic standards both implemented.

Section C: Backend & Server Communication

Responsible: Ravindu Sathsara

Deliverable: D2

C.1 Overview

The backend of Sentinel Guard consists of two local Python servers acting as components. The main processing server handles data from the Paho MQTT library to retrieve sensor data packets from the Mosquitto broker. The separate user management server uses FastAPI to execute administrative processes, providing authenticated REST API endpoints to be called by the web dashboard. Both services communicate with Firebase through the Firebase Admin SDK using the service account key for authentication. The Admin SDK bypasses Firestore security rules allowing for back-end access to write to Firestore for logging and storing data without client-level authentication.

C.2 Server Processing Pipeline

When the ESP32 publishes an encrypted packet to the MQTT broker, the Python server processes it through the following ordered pipeline:

- **Step 1 - Receive:** MQTT subscriber receives the packet from the Mosquitto broker on the subscribed topic.
- **Step 2 - DDoS check:** packet count within a defined time window is evaluated. Packets exceeding the rate threshold are discarded and logged as a DDoS event.
- **Step 3 - Structure validation:** packet format and required fields are verified. Malformed packets are rejected before any cryptographic processing.
- **Step 4 - HMAC-SHA-256 verification:** the server recomputes the HMAC digest over the received payload and compares it to the transmitted digest. Any mismatch causes immediate rejection and a tampered packet event is logged to Firestore.

- **Step 5 - Replay attack check:** the packet sequence number is compared against the last accepted value. Duplicate or out-of-order sequence numbers are rejected and logged as replay attack events.
- **Step 6 - AES-128 decryption:** the validated, authenticated payload is decrypted using the pre-shared AES-128 key.
- **Step 7 - Sequence validation:** internal sequence numbering within the decrypted payload is cross-checked for consistency.
- **Step 8 - Range check:** decrypted temperature and humidity values are validated against DHT22 physical limits (0–50 °C, 20–70 % RH).
- **Step 9 - ML anomaly detection:** the reading is forwarded to the XGBoost classifier. The model returns a classification (normal or anomaly) and a confidence score.
- **Step 10 - Firestore write:** sensor data is written to the sensor_readings collection. If the ML result is anomaly, a security event is written to the security_events collection with the classification, confidence score, device ID, and out-of-range flag.

C.3 User Management API (FastAPI)

REST API endpoints exposed by the separate FastAPI server enables web dashboard calls to REST API endpoints with Bearer tokens for administrative operations required by the web application; for example, GET /admin/users returns a list of admins, POST /admin/users invites new users, and PATCH /admin/users/{uid}/role updates an existing user's role. Under this design model, the FastAPI server verifies the Bearer token, then uses the Firebase Admin SDK to interact with Firebase Authentication and Firestore to process the request. This design maintains separation of concerns; the MQTT Data Pipeline is separate from the User Management System's API.

C.4 Firestore Data Structure

- The backend writes to the following Firestore collections:
- sensor_readings such as timestamped temperature, humidity, device ID, packet status, and sequence ID for each accepted reading.
- security_events - one document per detected security event. Fields: acknowledged, deviceId, eventType, message, mitigated, mitigationAction, mlResult, severity, timestamp, rawPayload. Events include anomaly detections, tampered packets, replay attacks, DDoS events, and processing errors.

- `system_logs` - system activity log written by the backend. Fields include log level (ERROR, WARN, INFO, DEBUG), event description, and timestamp. This is the collection the System Logs UI page reads from.
- `audit_logs` - admin-read-only collection; no client writes permitted. All entries written via Admin SDK. Captures security-relevant events for compliance and audit purposes.
- `users/{uid}` - user profile documents. Fields: role (admin / standard), email, lastLogin. Written by the backend on account provisioning; read by the frontend for RBAC cross-check.
- `backup_metadata` - admin read/write only. Present in Firestore rules, indicating the backup system is partially planned. Full automated backup is not implemented in the current version (see NFR-7.1).

C.5 Audit Logging

Audit log entries are written directly to Firestore from the Python backend using Admin SDK. The event types that are recorded in the logs include: tampered packets (failed HMAC), replay attack (duplicate sequence ID), DDoS event (rate limit exceeded), false data injection attempts, machine learning anomaly detection, and processing errors. The `audit_logs` and `system_logs` collections in Firestore have security rules that only allow admin-level users to read from the collections and prohibit clients from writing to either collection. This ensures that the integrity of the audit log records cannot be compromised from the frontend.

C.6 Requirements Satisfied

- **FR-15:** Firebase Admin SDK communicates with Firestore over HTTPS/TLS. All server-to-cloud traffic is encrypted.
- **FR-15:** TLS enforced for all server-to-cloud (Firebase) and server-to-broker (MQTTS) communications.
- **FR-23.1 / FR-23.2:** Audit log written to `system_logs` and `audit_logs` Firestore collections by the backend; Admin-only read access enforced by Firestore security rules.
- **NFR-7.1:** `backup_metadata` collection present in Firestore rules; full automated backup not implemented. Identified as a known limitation and future improvement.

Section D: ML Classification & Anomaly Detection

Responsible: Julian Mendis

Deliverable: D6

D.1 Model Overview

The incoming sensor reading is classified using an XGBoost classifier as either normal or an anomaly. XGBoost was chosen due to its excellent performance on tabular data, ability to resist overfitting because of a combination of regularization and gradient boosting, and ability to perform real-time inference on a local server. The model was trained on actual ESP32/DHT22 sensor data from Colombo during January – July of 2024, with a sample frequency of every 5 minutes.

D.2 Training Dataset

- **Total readings:** 105,351 (January - July 2024, 5-minute intervals).
- **Class distribution:** 102,223 normal (97.03 %) vs 3,128 anomaly (2.97 %).
- **Class imbalance handling:** SMOTE (Synthetic Minority Over-sampling Technique) applied to balance the training set.
- **Labels:** Assigned via hard temperature bounds rather than manual annotation.

D.3 Feature Engineering

The model uses 10 engineered features derived from the raw sensor readings:

- temp, humidity - raw sensor readings.
- rolling_mean, rolling_std - rolling statistics capturing temporal patterns.
- temp_diff - rate of change between consecutive temperature readings.
- hour, day_of_week, month - temporal features capturing diurnal and seasonal patterns.
- humidity_temp_ratio - derived feature capturing the relationship between the two sensor channels.
- sensor_encoded - encoded sensor identifier for multi-node scenarios.

D.4 Model Evaluation Results

Table D-1 presents the per-class classification metrics on the held-out test set of 15,803 samples.

Table D-1: XGBoost Classification Metrics (Test Set, n = 15,803)

Class	Precision	Recall	F1 Score
Normal	0.997	0.989	0.993
Anomaly	0.710	0.904	0.795

Overall: F1 = 0.795, ROC-AUC = 0.997 on the test set. 45 real anomalies missed; 173 false alarms.

The model demonstrates excellent overall classification performance, with a strong discrimination rating (ROC-AUC = 0.997). The classification accuracy for normal readings is also extremely high (F1 = 0.993). The F1 score for the anomaly class at 0.795 falls below the target for the NFR-05 (0.90). This is due to having a heavily imbalanced dataset and utilizing hard temperature cutoffs to label anomalies instead of using expert opinions to create the ground truth. The positive effect of utilizing SMOTE is supported by a high recall rate (0.904) for the anomaly class. The model has correctly identified 90.4% of the actual anomalies but has incurred 173 false alarms. Given that missing a true anomaly may carry a greater cost than having a false alarm when monitoring for security, this level of tradeoff between recall and precision provides sufficient justification for using this analytic technique for this purpose.

D.5 Mitigation Response

The event is recorded in the Firestore security_events collection when an anomaly occurs with the model's classification of the reading, including temperature, machine learning confidence score and out of range indicator. As of right now, the alert-only mitigation response does not include automated node isolation or IP blocking that is solely based on the ML classifier's findings. However, Administrators have access to manual mitigation controls which can be utilized via the web dashboard; these controls include Isolate Node, Block Suspected IP and Reset HMAC Keys, and Force Re-auth.

D.6 Requirements Satisfied

- FR-9 - XGBoost classifier distinguishes normal and anomaly classes in real time.
- FR-10.1 - Anomaly events logged to Firestore with confidence score; alert-only automated response. Manual mitigation controls available (partial).

- NFR-05 - Anomaly F1 = 0.795 (below 0.90 target); normal F1 = 0.993; ROC-AUC = 0.997 (partial).
- NFR-06 - Alert logging is immediate; automated sub-3-second mitigation not implemented (partial).

Section E: Web & Mobile Dashboard (Frontend)

Responsible: Najath Najeem

Deliverable: D3

E.1 Overview

A web dashboard (using React/Vite) and a mobile application (using React Native/Expo) have been developed for the Sentinel Guard front-end, both of which are linked to an underlying Firebase back-end. The front-end authenticates users; enforces role-based access control; provides real-time telemetry from live sensors, real-time security events, and real-time data through Firestore listeners; shows results of ML classification; and enables administrators to take mitigation actions that normal users cannot access while storing audit logs that can be viewed only by administrators.

E.2 Authentication & Role-Based Access Control

Firebase Authentication handles user authentication using email/password and/or username/password logins (see Figure E-6). Upon a successful login, Firebase generates a signed JWT (JSON Web Token) and returns it to the client. The front-end will determine a user's role based on the contents of the JWT's custom claim role field and by checking that value against the Firestore's users/{uid} document — if both values match, the user's role is stored in the global AuthContext. This process will prevent a user's role from being out-of-date or containing invalid claims and provide the user with possible access to the system than is warranted.

A ProtectedRoute component wraps all routes except the /login route; if the route required on the backend is Restricted and the resolved role doesn't match (admin), the user is then redirected to /dashboard. The routes accessible by Standard Users are only /dashboard, /sensorData, and /alertFeed: the Admin and System Log routes will not be listed in their sidebar nor be accessible from a direct URL. Session Termination will be signed out via Firebase ID Token and using Firebase Auth's token auto-expire function – after the ID token can't be silently refreshed upon authentication state changes, onAuthStateChanged will fire null event, clear AuthContext, and the user redirected to /login (NFR-9.1).

E.3 Dashboard Pages

- Dashboard (/dashboard) - Four summary cards: System Status, Active Alerts, HMAC Failures, and Uptime. Live temperature and humidity charts for Node 01 update via onSnapshot. Recent Alerts panel with unread count (Figure E-1).
- Sensor Data (/sensors) - Current readings with 'Within normal range' indicator; historical chart with 1h/6h/24h selector (Figure E-2). Satisfies FR-21.1
- Alert Feed (/alerts) - Security event table: Time, Event, Node, Severity, ML Result, Action. Filterable by All/Unread/Critical/Warnings; per-event acknowledgement (Figure E-3). Satisfies FR-10.3, FR-11.1, FR-12.1, FR-22.1
- User Management & Mitigation Controls (/admin, Admin only) - User table with role edit and force logout. Mitigation buttons: Isolate Node 01/02, Block Suspected IP, Reset HMAC Keys, Force Re-auth. Security layer status cards: AES-128, HMAC-SHA256, MQTTS/TLS, HTTPS/TLS, Nonce Protection, ML Classifier (Figure E-4). Satisfies FR-10.2, FR-13,FR-14.
- System Logs (/logs, Admin only) - Level filter (ALL/ERROR/WARN/INFO/DEBUG), keyword search, CSV export. Firestore listener implemented; collection name pending backend confirmation (Figure E-5). Satisfies FR-23.1, FR-23.2.

Figure E.1: Dashboard

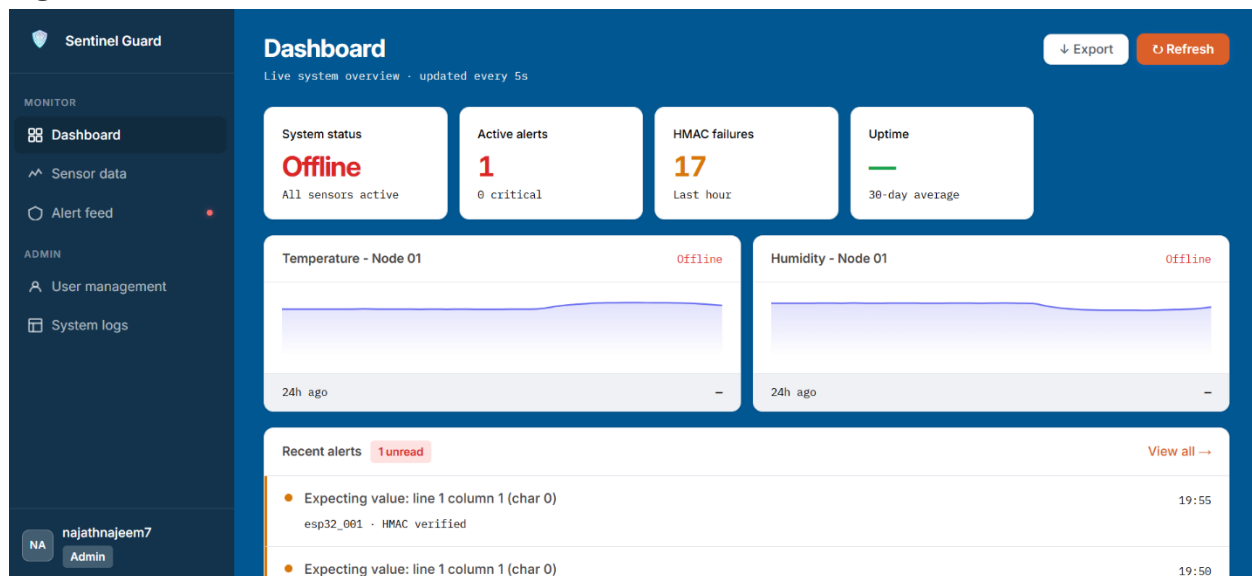


Figure E.2: Sensor Data

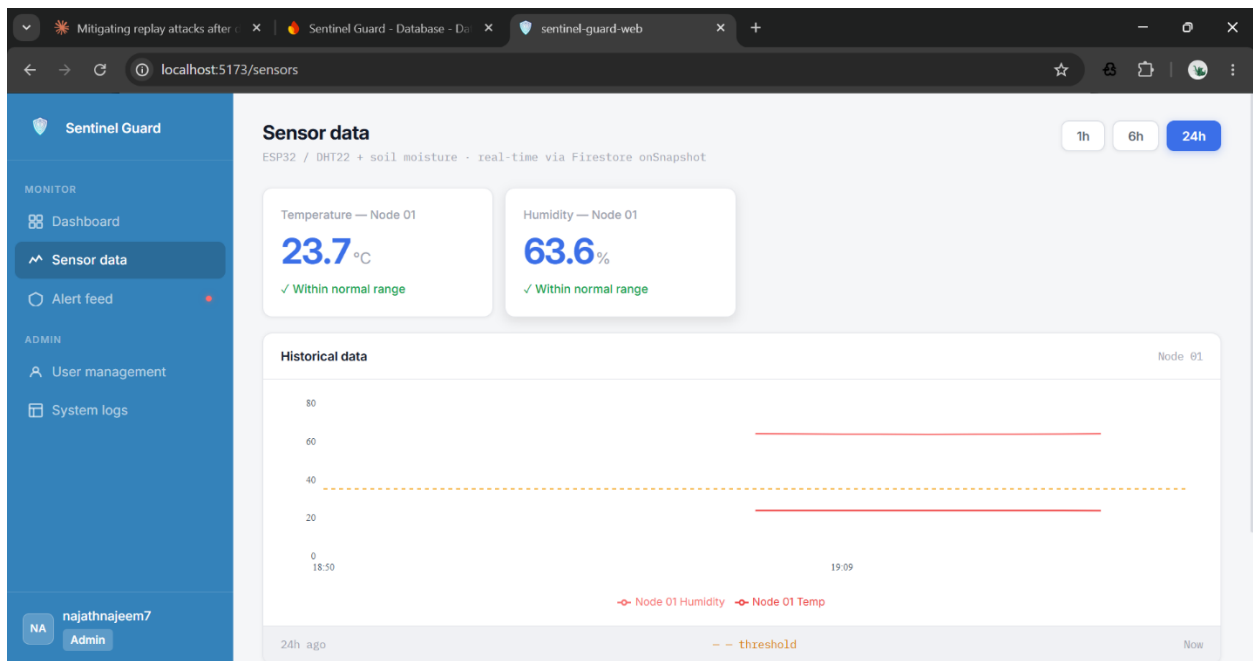


Figure E.3: Alert Feed

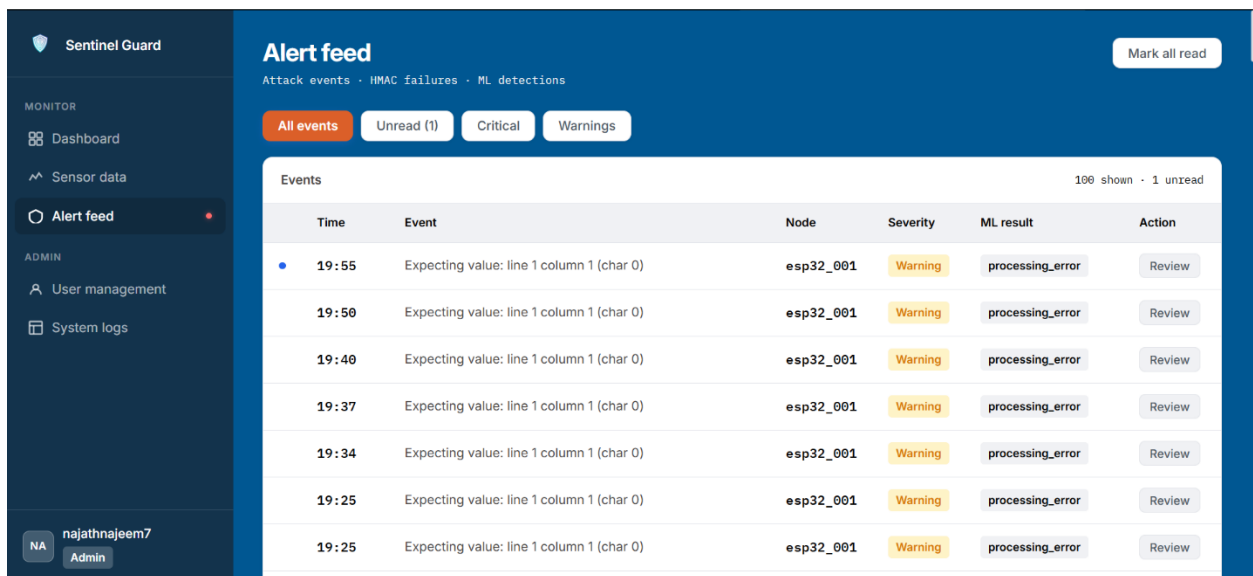


Figure E.4: Admin page

The screenshot displays the Sentinel Guard Admin interface. The left sidebar contains navigation links for MONITOR (Dashboard, Sensor data, Alert feed) and ADMIN (User management, System logs). The main content area is titled "User management" and includes a breadcrumb trail: "Firebase Auth · RBAC roles · Admin access only". A "+ Add user" button is in the top right. Below the header, there's a "Users" section with a table listing two users: "najathnajeem7@gmail.com" (Admin role) and "Ravindu" (Standard role). Each user has "Edit role" and "Delete" buttons. Below the table is a "Mitigation controls" section with a breadcrumb trail: "Admin only · Irreversible". It features buttons for "Isolate Node 01", "Isolate Node 02", "Block suspected IP", "Reset HMAC keys", and "Force re-auth all users". At the bottom, there are six status boxes: "AES-128" (Active), "HMAC-SHA256" (Active), "MQTT / TLS" (Active), "HTTPS / TLS" (Active), "NONCE PROTECTION" (Active), and "ML CLASSIFIER" (Running).

Name	Email	Role	Last login	Actions
☐	najathnajeem7@gmail.com	Admin		<button>Edit role</button> <button>Delete</button>
☐	Ravindu	Standard		<button>Edit role</button> <button>Delete</button>

Mitigation controls: Admin only · Irreversible

Isolate Node 01 Isolate Node 02 Block suspected IP Reset HMAC keys Force re-auth all users

AES-128 Active HMAC-SHA256 Active MQTT / TLS Active HTTPS / TLS Active NONCE PROTECTION Active ML CLASSIFIER Running

Figure E.5: System Logs

The screenshot displays the Sentinel Guard System logs interface. The left sidebar is the same as in Figure E.4. The main content area is titled "System logs" with a breadcrumb trail: "Auth · HMAC · ML · MQTT · Firestore · Admin only". An "Export" button is in the top right. Below the header, there's a filter bar with buttons for "ALL", "ERROR", "WARN", "INFO", and "DEBUG", along with a "Search logs..." input field. The "Log entries" section shows a list of 288 entries. The visible entries are all "INFO" level logs from "esp32_001" stating "Sensor packet accepted", with timestamps ranging from 12/05/2026, 20:08:12 to 12/05/2026, 20:08:52. Each entry has a "system" source.

System logs: Auth · HMAC · ML · MQTT · Firestore · Admin only

Export

ALL ERROR WARN INFO DEBUG Search logs...

Log entries: 288 entries

Timestamp	Level	Source	Message	System
12/05/2026, 20:08:52	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:49	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:48	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:43	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:40	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:37	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:34	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:31	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:28	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:25	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:22	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:19	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:16	INFO	esp32_001	Sensor packet accepted	system
12/05/2026, 20:08:12	INFO	esp32_001	Sensor packet accepted	system

E.4 Mobile Application

Real-time sensor and alert data can be accessed via Firestore using the same Firebase Project as the web dashboard; in this case, both applications are using the same Firestore database and application-level network connection, but React Native Expo uses "onSnapshot" listener functionality. The following AuthContext and RBAC enforcement will

also mirror those in the web application giving APIs that are role-based across both App(s) (FR-13).

E.5 Requirements Satisfied

Table E-1: Frontend Functional Requirements Traceability

FR	Requirement	Frontend Implementation	Status
FR-13/14	RBAC & authenticated access	Role resolved from Firebase JWT claim + Firestore users/{uid} cross-check. All routes in ProtectedRoute; /admin and /logs redirect non-admins.	Met
FR-19.1/20.2	Login / Logout	Firebase Auth email/password. signOut() clears session and redirects to /login immediately.	Met
FR-10.3	ML classification display	Alert Feed renders per-event ML result badge (attack / anomaly / normal / false_pos / processing_error).	Met
FR-10.2	Mitigation controls	Admin page: Isolate Node 01/02, Block Suspected IP, Reset HMAC Keys, Force Re-auth (Admin only).	Met
FR-11.1/12.1	Real-time alert notification	Firebase onSnapshot on security_events drives instant UI update; unread dot on nav item.	Met
FR-21.1	Historical sensor data	Sensor Data page: 24 h chart with 1h/6h/24h selector, sourced live from Firestore.	Met
FR-22.1	Acknowledge alert	Per-event acknowledge writes back to Firestore; acknowledged events visually distinguished.	Met
FR-23.1/23.2	Audit log (Admin only)	System Logs route is Admin-only; level filter, keyword search, CSV export implemented.	Met

FR	Requirement	Frontend Implementation	Status
		Firestore collection name pending confirmation from backend.	

Section F: Non-Functional Requirements Evaluation

Table F-1 evaluates each non-functional requirement against the evidence from the deployed system.

Table F-1: Non-Functional Requirements Evaluation

NFR	Requirement	Evidence	Status
NFR-03	Sensor data within 5 s	Firestore onSnapshot delivers updates within 1–2 s under normal conditions. Dashboard confirms 'updated every 5s'.	Met
NFR-02	AES-128 & HMAC-SHA-256	Both implemented at the application layer in addition to TLS, satisfying the dual-mechanism cryptographic requirement.	Met
NFR-05	ML accuracy $\geq$ 90%	XGBoost overall F1 = 0.795; anomaly class F1 = 0.795. Normal class F1 = 0.993. ROC-AUC = 0.997. Anomaly F1 below 90% threshold; SMOTE and hard-label assignment noted as limiting factors.	Met
NFR-06	Mitigation within 3 s	Anomaly mitigation is alert-only (no automated blocking). Manual admin controls are available but no automated sub-3-second response is implemented for anomaly events.	Partial
NFR-7.1	Scheduled backup	backup_metadata collection present in Firestore rules confirming backup system is planned. Full automated backup not	Met

NFR	Requirement	Evidence	Status
		implemented in current version. Identified as a known limitation and future improvement.	
NFR-08	Browser compatibility	React/Vite build targets modern ES modules. Confirmed functional in Chrome. No specialist plugins required.	Met
NFR-9.1	Session timeout	Firebase ID token 1-hour expiry; onAuthStateChanged fires on expiry; user redirected to /login.	Met
NFR-10	Usability	Dashboard uses plain-language labels and badge-based status indicators. Admin controls labelled as irreversible. No specialist training required to interpret dashboard output.	Met

Conclusion

The Sentinel Guard IoT security system offers a fully unified solution across four distinct systems (or layers). The IoT node collects and validates sensor data (to ensure accuracy) prior to transmitting (sending) over an encrypted and authenticated communications path (connection). The security layer comprises an array of security mechanisms which when combined provide several levels of protection and meet all standards required for secure communications. The XGBoost ML classifier is able to achieve excellent performance as measured by the ROC-AUC score of 0.997 which indicates a very high level of ability to differentiate between the normal and the anomalous behaviour of an IoT system. It was also able to detect 90.4% of all anomalies (malicious activity). In addition to the ML classifier, the alert logs generated for all alerts are pushed to Google Cloud Firestore for further review/mitigation by the Administrator. Both the web and mobile interfaces provide real-time information for monitoring both Sensor data and Security Events with enforced Role Based Access Control.

Three areas remain not fully addressed; the F1 score for the Anomaly Class is only 0.795 which is lower than the NFR target (0.90) due primarily to data set imbalance and use of hard-label annotations; there are currently no automated responses to ML-detected Anomalies (i.e. Automated Mitigation of less than 3 seconds) currently exist, only Alert and Manual Control option for Administrators; there is a Backup plan created using the backup_metadata Firestore collection but it is not yet fully automated. All of these gaps are well-identified and the associated functionality has been either implemented in the codebase or is being recommended as a priority for production deployment.